All Things Agentic Hackathon https://allthingsagentichackathon.devpost.com/
https://allthingsagentichackathon.devpost.com/rules
https://allthingsagentichackathon.devpost.com/resources

Ignore the deadline and how much time we have.

You are acting as an elite hackathon strategist, product inventor, systems researcher, open-source founder, and Google-level agentic AI architect.
Your task is NOT to give me ordinary hackathon ideas.
Your task is to discover 5 genuinely exceptional project ideas for the All Things Agentic Hackathon by Google:
https://allthingsagentichackathon.devpost.com/
Resources: https://allthingsagentichackathon.devpost.com/resources
Rules: https://allthingsagentichackathon.devpost.com/rules
I want ideas with the potential to be:

- Grand Prize level
- technically impressive even to Google engineers
- genuinely useful outside the hackathon
- strong enough to become a real open-source project/startup
- something people would actually install/use
- potentially the kind of GitHub project that could become extremely popular
- difficult to dismiss as “just another LLM wrapper”
- memorable enough that judges remember it after seeing hundreds of submissions

Do NOT optimize for being easy.
Optimize for:
novelty × usefulness × agentic depth × technical elegance × demo impact × open-source potential × hackathon fit.

STEP 1 — RESEARCH BEFORE BRAINSTORMING

Before generating ideas, deeply research the hackathon.
Read the official:

- overview
- rules
- resources
- judging criteria
- track descriptions
- required Google technologies
- bonus scoring
- examples organizers give
- workshops/topics organizers emphasize

Pay special attention to what Google appears to WANT participants to demonstrate.
The hackathon requires:

- Gemini 3.5 or newer through Gemini API or Vertex AI
- at least one Google agent framework:
    - Google ADK
    - GenAI SDK
    - Antigravity SDK
    - GenKit
- at least one Google Cloud infrastructure service:
    - Cloud Run
    - Firestore
    - Pub/Sub
    - Cloud SQL
    - GKE
    - etc.

Tracks:

Taskmaster

Complete autonomous workflows.
Agents should notice events, make decisions, take actions, interact with systems, handle multi-step workflows, and finish work without constant human guidance.

Collaborative Partner

Persistent, adaptive agents that collaborate with users over time, ask useful clarifying questions, learn from feedback, maintain context/memory, and change how they work based on the individual.

Fortified Enterprise Fleet

Networks of production-grade agents using concepts such as:

- Agent Registry
- Agent Runtime
- Memory Bank
- Agent Identity
- Agent Gateway
- Model Armor
- Agent Observability
- long-running operations
- security
- governance
- persistent state
- multi-agent orchestration
- data sovereignty

- cross-department agent discovery

Important judging weights:

- 40% Innovation & Operational Utility
- 30% Architectural Discipline & Tech Stack
- 30% Demo & Production Readiness

Read the detailed rules because they contain stronger hints than the marketing page.
Look for wording around:

- autonomous execution
- “twist”
- unusual problems
- unusual users
- messy/unstructured data
- evolving knowledge
- active mutation of state/data
- asynchronous workflows
- multi-agent delegation
- failure recovery
- persistent state
- security
- proof of action
- production readiness

Use those details strategically.

STEP 2 — RESEARCH WHAT ALREADY EXISTS

This part is extremely important.
Do NOT assume an idea is new because you personally have not heard of it.
For every promising direction, search:

- Google
- GitHub
- Product Hunt
- Hacker News
- Reddit where useful
- startup databases/results
- arXiv/research papers
- Google research
- current agent frameworks
- 2025–2026 products
- YC/startup launches where relevant

Search for:

- existing commercial products
- open-source projects
- recent research papers

- GitHub repositories
- previous hackathon projects
- agent protocols
- MCP/A2A projects
- Google Agent Platform functionality

If an existing project already does substantially the same thing:
REJECT the idea.
Unless you can introduce a fundamental difference that changes the product category or capability.
Changing:
“AI agent for lawyers”
into:
“AI agent for doctors”
is NOT innovation.
Changing UI, model provider, target industry, or adding multiple agents is NOT enough.

HARD BAN LIST

Do NOT recommend any of the following unless you discover an extraordinarily different underlying invention:

- AI video generation/editing
- automated coding
- coding agents
- automated debugging
- IDE agents
- Cursor/Windsurf/Claude Code/Codex/Antigravity competitors
- generic browser agents
- generic computer-use agents
- generic “AI controls your computer”
- OpenClaw clones
- Hermes clones
- general personal assistants
- everyday reminder agents
- “remember everything about me”
- email assistants
- inbox agents
- meeting summarizers
- tutors
- study assistants
- generic RAG
- generic research agents
- generic workflow builders
- Zapier-with-AI

- n8n-with-AI
- simple MCP aggregators
- simple A2A aggregators
- generic agent dashboards
- “one interface for every AI”
- model routers
- prompt routers
- generic multi-agent orchestration
- AI CRM
- AI project manager
- AI product manager
- AI receptionist
- AI customer support
- AI calendar assistant
- AI note-taking
- resume agents
- generic finance agents
- generic healthcare chatbots
- generic legal-document assistants
- “upload document → ask questions”
- dashboards that mainly display AI output
- basic alerting systems
- systems that only recommend what a human should do
- LLM + API + UI wrappers

If the central pitch can be summarized as:
“We use Gemini to understand something and then call an API,”
it is probably too weak.
Reject it.

WHAT I AM LOOKING FOR

Think more like the invention of:

- Docker
- Git
- Kubernetes
- BitTorrent
- Figma multiplayer
- Shazam
- OpenClaw
- MCP
- A2A

Not literally those technologies.
I mean the category-creating quality.

I want something where people might say:
“Wait… why didn't this exist already?”
or:
“I didn't know agents could work like this.”
or:
“I need this.”
It can be:

- consumer
- developer
- prosumer
- enterprise
- government
- infrastructure
- creator
- science
- robotics
- physical world
- distributed systems
- productivity
- communication
- collaboration
- data
- security
- automation
- agent infrastructure
- entirely new category

B2C, B2B, B2B2C, B2G, open source, desktop, mobile, web, local, cloud — anything is allowed.
Do not artificially limit the search space.

THINK BEYOND “AN AGENT THAT DOES A JOB”

Strongly explore entirely new primitives for autonomous software.
Examples of TYPES OF QUESTIONS to explore — these are NOT ideas to copy:

- What fundamental capability do autonomous agents still lack?
- What assumptions from normal software break when software becomes autonomous?
- What new operating-system abstractions will agents require?
- What does an “agent-native” application look like instead of adding an LLM to an old application?
- What happens when thousands of agents exist instead of one?
- How should autonomous systems coordinate without central control?
- How should agents recover when the world changes after they acted?
- What new kind of networking/protocol/state/memory/permission/transaction abstraction becomes necessary?

- What can agents do asynchronously that ordinary software cannot practically do?
- What workflows are currently impossible because they cross too many systems, people, devices, time periods, or data formats?
- What could become a protocol/ecosystem rather than a single application?
- What could other developers build on top of?
- What could become an open-source infrastructure project?
- What changes when an agent continues operating for days/weeks instead of answering one request?
- What problems become possible when Gemini can understand multimodal messy state and agents can take actions?
- Can autonomous systems create something emergent that no single model/agent currently provides?

Also explore ideas where:
event → reasoning → action → verification → adaptation → continued execution
is fundamental.
Not:
prompt → answer.

OPEN-SOURCE / VIRAL PRODUCT TEST

At least some candidates should have the potential to become a project people install themselves.
Ask:
Would developers realistically run:
pip install ...
or:
npm install ...
or:
docker compose up
because this gives their agents something fundamentally useful?
Could it become infrastructure other agent projects depend upon?
Could third parties build plugins/extensions/agents/modules for it?
Could an ecosystem form?
Could somebody build something with it that the original creator never anticipated?
That is much more interesting than a single-purpose SaaS demo.

HARDWARE

Hardware is allowed, but the entire prototype should ideally cost under $500.
Do NOT force hardware into an idea just to appear impressive.
Use hardware only when:

- the physical-world interaction is central to the innovation
- software-only cannot deliver the same wow factor
- hardware creates an unusually strong live demo
- people would genuinely use it
- it is practical for a hackathon prototype

Cheap hardware possibilities can include:

- ESP32
- Raspberry Pi
- microphones
- webcams
- inexpensive sensors
- BLE
- NFC
- environmental sensors
- simple motors/servos
- USB devices
- consumer electronics

But software should win unless hardware makes the idea dramatically stronger.

GOOGLE TECHNOLOGY MUST MATTER

Do NOT bolt Google Cloud onto the idea at the end.
For each finalist, Gemini/ADK/Google Cloud should be architecturally necessary.
Strong uses could include:

- Gemini 3.5 multimodal reasoning
- Google ADK multi-agent orchestration
- long-running Agent Runtime
- Memory Bank
- Agent Registry
- Agent Identity
- Agent Gateway
- Model Armor
- Agent Observability
- A2A
- Pub/Sub event-driven execution
- Firestore persistent distributed state
- Cloud Run
- Vertex AI
- GKE if actually justified

If you removed Gemini/agents and the product would work almost identically with ordinary deterministic code, question whether it belongs in this hackathon.
Likewise, don't make everything AI just because it is an AI hackathon.
Use deterministic software where deterministic software is superior.

Use agents specifically where reasoning, ambiguity, adaptation, planning, delegation, long-running autonomy, multimodal interpretation, or dynamic tool selection are necessary.

BRAINSTORMING PROCESS

Internally generate at least 30–40 serious ideas.
Do NOT output all 30–40.
For each candidate, internally evaluate:

Novelty

Does something substantially equivalent already exist?

Need

Would actual people care?

Frequency

Would anyone use it repeatedly?

Agent necessity

Does autonomy genuinely improve the product?

Workflow depth

Is there a meaningful multi-step workflow?

Google fit

Does it showcase Google's agent stack naturally?

Architecture

Can it demonstrate state, memory, events, resilience, security, or multi-agent behavior?

Demo

Can a judge understand the magic in under 30 seconds?

Four-minute proof

Can we visibly demonstrate real actions rather than claims?

Expansion

Can this grow much larger than the hackathon prototype?

Open-source potential

Could people build on top of it?

Defensibility

Would copying the UI and connecting Gemini reproduce 80% of it?
If yes, reject it.

Clone difficulty

Is there meaningful systems engineering, protocol design, architecture, data structure, distributed execution, unique workflow engine, or ecosystem design underneath?

“Google engineer reaction”

Would a strong Google engineer see something technically interesting?

“Normal user reaction”

Would someone who doesn't care about AI still understand why it's useful?
The strongest ideas should satisfy BOTH.

RED TEAM EACH FINALIST

Before choosing the final five, attack each candidate.
Ask:
Isn't this just X?
Doesn't company Y already do this?
Couldn't OpenClaw do this with a prompt?
Couldn't Zapier/n8n reproduce this?
Couldn't Google build this with existing Agent Platform features?
Is this just MCP/A2A with a UI?
Why does this need an agent?
Why would anyone install this?
Is this solving an invented problem?
Is the architecture actually necessary?
Would the four-minute demo be boring?
What makes this more than a weekend toy?
What prevents another contestant from making the same idea?
If you cannot strongly answer those objections:

remove the idea.

DIVERSITY

Do not give me five versions of one concept.
The final five should explore meaningfully different categories.
Ideally include a mixture such as:

- one possible new open-source agent primitive
- one exceptional Taskmaster product
- one genuinely new Collaborative Partner concept
- one sophisticated Fleet architecture
- one wildcard that could potentially beat everything else

But quality is more important than filling categories.
If the five strongest ideas are all in one track, say so.

SCORING

Privately score each candidate from 0–10 on:

- originality
- usefulness
- agentic necessity
- autonomy
- technical depth
- Google-stack fit
- architecture
- demo wow factor
- open-source/star potential
- startup potential
- defensibility
- judge memorability

Use weighted scoring:
20% Originality
18% Real usefulness / demand
15% Agentic necessity
12% Demo impact
10% Google stack fit
10% Architecture depth
5% Open-source potential
5% Defensibility
5% Expansion/startup potential
Then apply the hackathon's official 40/30/30 judging criteria as a second evaluation.

Do NOT call something “95% chance to win.”
No one can honestly know that.
Instead give a Hackathon Fit score /100.

FINAL OUTPUT

Do NOT show me the 30–40 brainstormed ideas.
Only output the 5 absolute strongest finalists after research, elimination, comparison, novelty checking, and red-teaming.
Rank them:
#1 → #5.
For each idea provide:

NAME

Use a temporary but strong memorable product name.

ONE-SENTENCE PITCH

One sentence a judge instantly understands.

THE NEW THING

Explain the actual invention.
Not marketing.
What exists after we build this that did not exist before?

HOW IT WORKS

Give the actual agentic workflow.
Show:
event/input
→ agents/reasoning
→ tools/actions
→ persistent state
→ verification
→ adaptation
→ completion
Be specific.

WHY PEOPLE WANT IT

Who would realistically use/install it?
Why repeatedly?
What pain disappears?

WHY IT ISN'T A WRAPPER

Explicitly explain the systems/architecture/product innovation.

EXISTING CLOSEST THINGS

Name the closest products, GitHub projects, papers, protocols, or startups you found.
Then clearly explain:
Existing X does A.
Our idea does B.
The fundamental difference is C.
If the differentiation is weak, the idea should never have reached the final five.

GOOGLE STACK

Specify exactly how to use:

- Gemini 3.5+
- ADK / GenAI SDK / Antigravity / GenKit
- Google Cloud

And optionally:

- A2A
- Agent Registry
- Agent Runtime
- Memory Bank
- Agent Identity
- Agent Gateway
- Model Armor
- Agent Observability
- Gemma

Every Google component must have a real reason to exist.

TRACK

Choose:

- Taskmaster
- Collaborative Partner
- Fortified Enterprise Fleet

Explain briefly why.

4-MINUTE KILLER DEMO

Design the exact live demo.
I want something visually undeniable.
The demo should contain a moment where the judge thinks:
“Whoa.”
Prefer state changing on screen, live agents acting, systems adapting, failures being recovered from, devices reacting, workflows restructuring, or something similarly undeniable.
Avoid demos that are primarily somebody chatting with an AI.

OPEN-SOURCE / ECOSYSTEM POTENTIAL

Explain whether other people could build on top of it and why they would.

HARD PART / MOAT

What technically difficult component makes this non-trivial to clone?

BIGGEST RISK

What could make the project fail or seem less novel?

SCORE

Give:

- Hackathon Fit /100
- Originality /10
- Utility /10
- Agentic Depth /10
- Demo /10
- Architecture /10
- Open-source Potential /10

AFTER THE FIVE

Finish with only three short sections:

MY PICK

Choose exactly one project you would build if your own goal were to maximize the probability of winning the Grand Prize.
Explain why.

MOST VIRAL

Choose the one with the strongest chance of becoming a widely used/open-source project independent of the hackathon.

MOST IMPRESSIVE TO GOOGLE ENGINEERS

Choose whichever contains the most interesting technical/agent-systems innovation.
These can be the same idea.

FINAL QUALITY BAR

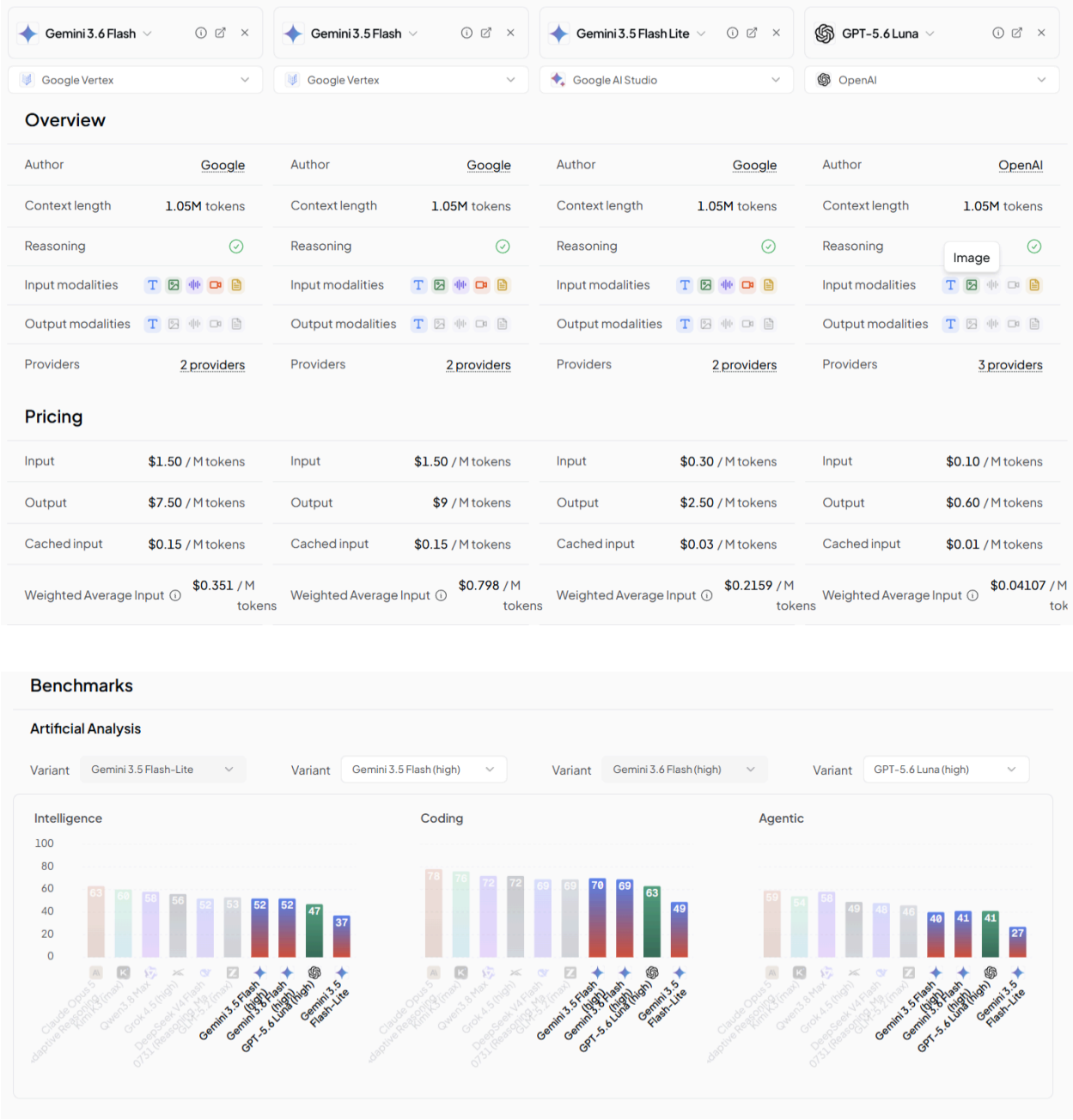
Do not give me ideas merely because they are feasible.
I can build something difficult.
Do not optimize around a weekend.
Do not give safe ideas.
Do not give generic “AI-powered” products.
Do not give ideas where the AI is the product.
I want the system/product/protocol/workflow invention to be the product and Gemini/agents to make it possible.
Think until you have something where you genuinely believe:
“This could become a real new project people care about even if there were no hackathon.”
Then select the five.
If your first ideas resemble common hackathon entries, discard them and brainstorm deeper.
Do the research first. Generate 30–40 candidates internally. Be extremely skeptical. Only show me the five ideas that survive.

Final/Testing Model (Support Input: Text, Image, Audio, Video, File) (Through Google Vertex, Google AI Studio)

1. Gemini 3.5 Flash Lite
2. Gemini 3.5 Flash
3. Gemini 3.6 Flash

Testing/Debugging Model

1. GPT 5.6 Luna



Build next-generation AI agents on Gemini and Google Cloud. Most AI today waits for you to ask. The next generation doesn't. AI agents are systems that can take a goal, make a plan, and actually carry it out — pulling information, making decisions, and completing multi-step tasks on their own, while you do something else. One of the famous examples is the project OpenClaw and Codex “/goal”. All Things Agentic hackathon is a global hackathon that challenges you to build one. Using Gemini, Google's open-source Agent Development Kit (ADK), and Google Cloud, you'll create an agent that takes real action to remove everyday friction — at work, at home, or across an entire enterprise.

Focus mainly on Google Use gemini, Google cloud console, google's opensources or google's agent development kit ADK, or any google related. Testing using other providers is allowed, but you need to be flexible to change.

Requirements (What to Build)

Build and deploy a next-generation, autonomous AI Agent leveraging Gemini 3.5 Flash (Or other Gemini models) that operates beyond standard chat loops. The system can run asynchronously in the background, handle the heavy lifting of complex workflows, or dynamically manipulate data pipelines and representations.

Projects must be built within one of these three categories:

1. Taskmaster: Build a complete workflow, not just a chatbot, not just an AI wrap. Don't just make an agent that writes text. Make one that takes action. Find a messy, multi-step chore in your job, classes, or personal life. Build an agent that handles the details, sends the right info to the right places, and proves it can do the heavy lifting for you- something that doesn't need a human in the loop or any human to interact at any time.
2. Collaborative Partner: Build an agent that leads the way and takes notes. It should ask clarifying questions, guide the user step-by-step, and have a clear way to capture feedback, so it constantly adapts to the user's unique way of thinking.
3. Fortified Enterprise Fleet: Build a scalable network of institutional agents that hook into official enterprise infrastructure. Teams must demonstrate how agents are cataloged for cross-department use, how they safely maintain context across weeks of asynchronous operations, and how they interact with production data without violating enterprise compliance, data sovereignty, or security policies.
    a. Discovery & Lifecycle: Agent Registry (the central repository for publishing, versioning, and discovering enterprise-approved agents).
    b. Core Execution & State: Agent Runtime (for long-running, asynchronous background execution) and Memory Bank (for persistent, secure cross-session context over extended timelines).
    c. Security & Governance: Agent Identity (For zero-trust access control), Agent Gateway (for unified routing and policy enforcement), and Model Armor (inline guardrails to block prompt injection, tool poisoning, and PII leaks).
    d. Telemetry: Agent Observability (OpenTelemetry-compliant audit logs and end-to-end reasoning chain traces).

Every project, in every track, must use:

1. Gemini 3.5 or newer accessed through Gemini API or Vertex AI
2. At least one Google Agent Framework: Google ADK, GenAI SDK, Antigravity SDK or GenKit
3. At least one Google Cloud infrastructure service (such as Cloud Run, Cloud SQL, Firestore, GKE, Pub/Sub).

Note on cost & deployment: Your app does not need to be publicly accessible or live at the exact moment of submission or judging (so you don't rack up unnecessary costs). You just need to provide clear proof that it was built and deployed on Google Cloud — for example, shown in your demo video and code repository. See https://allthingsagentichackathon.devpost.com/resources for tips on keeping your costs near zero.

For Bonus Points, optionally you can do one or both of the following:

- Publish a piece of content (blog, podcast, video): Covering how the project was built on any public platform (e.g., medium.com, http://dev.to/, YouTube, etc.). The content must be public (not unlisted). You must include language that says you created the piece of content for the purposes of entering this hackathon.
- Publish a social media post: Highlight or promote your project on social media post on X, LinkedIn, Instagram, or Facebook. For any social media posts on platforms such as X or LinkedIn, include the hashtag #AllThingsAgenticHackathon.
- Successfully integrate Google AI models such as Gemma, Veo or Lyria.

Prizes

$180,000 in prizes

Grand Prize

$50,000 in cash
1 winner
• $50,000 in USD
• $5,000 in Google Cloud Credits for use with a Cloud Billing Account
• Virtual Coffee with a Google Team Member
• Social Promo

The Taskmaster

$20,000 in cash
1 winner
•$20,000 in USD
• $2,000 in Google Cloud Credits for use with a Cloud Billing Account
• Virtual Coffee with a Google Team Member
• Social Promo

The Collaborative Partner

$20,000 in cash
1 winner
• $20,000 in USD
• $2,000 in Google Cloud Credits for use with a Cloud Billing Account

• Virtual Coffee with a Google Team Member
• Social Promo

The Fortified Enterprise Fleet

$20,000 in cash
1 winner
• $20,000 in USD
• $2,000 in Google Cloud Credits for use with a Cloud Billing Account
• Virtual Coffee with a Google Team Member
• Social Promo

Startup Excellence (Incorporated Organizations eligible - see rules for details)

$20,000 in cash
1 winner
• $20,000 in USD
• $5000 in Google Cloud Credits for use with a Cloud Billing Account
• Virtual Coffee with a Google Team Member
• Social Promo
Must be submitting on behalf of an organization that is incorporated, and you must provide a corporate email address

Individual/Hobbyist (Best Team/Solo Build)

$10,000 in cash
2 winners
• $10,000 in USD
• $1000 in Google Cloud Credits for use with a Cloud Billing Account
• Virtual Coffee with a Google Team Member
• Social Promo

Best Architectural Design

$5,000 in cash
2 winners
• $5,000 in USD
• $1000 in Google Cloud Credits for use with a Cloud Billing Account

Best Multimodal UX

$5,000 in cash
2 winners
• $5,000 in USD
• $1000 in Google Cloud Credits for use with a Cloud Billing Account

Honorable Mentions

$2,000 in cash
5 winners

• $2,000 in USD
• $500 in Google Cloud Credits for use with a Cloud Billing Account

Judging Criteria

Innovation & Operational Utility- 40%
How much real-world friction does the agent remove on its own? We reward autonomous, high-value action over simple chat — agents that make decisions and complete tasks with little to no hand-holding.

Architectural Discipline & Tech Stack- 30%
How sound are your engineering choices? We look at how you decouple systems, manage state and memory, secure credentials, and handle failures — robust, production-minded agents, not brittle scripts.

Demo & Production Readiness- 30%
How clearly do your video and repo prove it works? We want a live, unedited demo, a clean architecture diagram, reproducible setup, and visible proof it runs on Google Cloud.

EXPLORE THE TRACKS

The Taskmaster — in depth

- The focus: An event-driven workflow with autonomous routing. Your system acts like a smart coordinator — watching for a change, figuring out what needs to happen next, and interacting with different apps to get the job done, from start to finish, without you guiding each step.
- Examples: An "Automated Product Manager" that reads meeting transcripts, extracts action items, creates Jira tasks, and posts a summary to Slack. A "Freelance Pipeline" agent that watches your inbox for new inquiries, checks your calendar, drafts a proposal from your past work, and saves it for review.

The Collaborative Partner — in depth

- The focus: Stateful, multi-turn dialogue with real-time context retrieval (RAG) and persistent memory, so your agent adapts and personalizes based on past interactions instead of starting over each time.
- Examples: An expert guide that helps you understand a dense legal document, quizzes you as you go, learns which concepts you struggle with, and adapts future explanations. A UI/UX helper for non-designers that turns a vague idea into a wireframe and learns your brand preferences from your corrections.

The Fortified Enterprise Fleet — in depth

- The focus: Corporate agent discovery, multi-agent orchestration at scale, long-term state persistence, runtime observability, and security posture enforcement. Show how an organization can discover your agents, audit their reasoning, trust their data handling, and scale them safely. Open to everyone — not just startups or enterprises.
- Recommended tech (Gemini Enterprise Agent Platform): Agent Registry (discovery/versioning); Agent Runtime (long-running async execution) + Memory Bank (persistent cross-session context); Agent Identity (zero-trust access), Agent Gateway (routing + policy), Model Armor (guardrails against prompt injection, tool poisoning, PII leaks); Agent Observability (audit logs + reasoning-chain traces).
- Example: An "Enterprise Supply Chain Orchestrator" that a procurement manager finds in the internal Agent Registry to run a multi-week vendor onboarding cycle — monitoring delivery webhooks, remembering negotiation data via Memory Bank, securely querying private ERP inventory with Agent Identity, coordinating with a logistics sub-agent through Agent Gateway, and screening all external email with Model Armor.

Gemini Enterprise Agent Platform (GEAP) — for the Fortified Enterprise Fleet track

The Gemini Enterprise Agent Platform is Google Cloud's managed platform for building, deploying, governing, and scaling enterprise AI agents. It's the recommended toolkit for the Fortified Enterprise Fleet track and includes the Agent Registry (a central catalog to securely store, discover, and govern agents and tools), Agent Runtime (a fully managed, scalable runtime for long-running agents), and Memory Bank (long-term, cross-session memory that personalizes agent interactions), plus identity, gateway, guardrails, and observability.

- Platform overview
- Documentation home
- Agent Runtime
- Memory Bank
- Announcement blog

BUILD YOUR AGENT

- Gemini API & Google AI Studio — models, quickstarts, multimodal guides
- Agent Development Kit (ADK) — the fastest way to build, evaluate, and deploy agents: github.com/google/adk-python
- Antigravity SDK — a pre-packaged agent runtime tightly integrated with Gemini:
- Genkit — open-source framework for AI-powered apps (JS, Go, Python):
- Cloud Run — deploy your agent with a URL; scales to zero when idle
- Firestore — simple NoSQL datastore for agent state/memory

Pick a problem worth solving

The strongest projects start from a real, specific annoyance rather than a clever technical idea. Find a messy, multi-step chore in your job, your classes, or your personal life — the kind of thing you do every week and resent. Then build the agent that handles it end to end.

That framing does double duty: it makes your demo obviously valuable, and it keeps you from building a chatbot by accident. Judges reward agents that take action, not agents that talk well.